Baladithya Balamurugan
1699864 bbalamur@ucsc.edu

CSE 102 (Fall 2020) – Homework 1

Due: Oct 15 at 10am PT on Gradescope.
Total: 50 points. Sub-parts of a question are equally weighted unless indicated otherwise.
Due to the large class size, only a subset of the problems may be graded and the total points adjusted accordingly.

Before you begin the assignment, please read the following carefully.

- Read the Homework Guidelines.
- Every part of each question begins on a new page. Do not change this.
- This does not mean that you should write a full page for every question. Your answers should be short and precise. Lengthy and wordy answers will lose points.
- Do not change the format of this document. Simply type your answers as directed.
- You are **not** allowed to work in teams.
- You **are** allowed to discuss homework problems at a high level with groups of up to 4 classmates, however any submitted work must be your own. You may not share your written solutions with one another.
- Indicate any collaborators on the line below, and in each solution specify any key ideas obtained from group discussion.

I have read and agree to the collaboration policy. – Baladithya Balamurugan, bbalamur@ucsc.edu
Collaborators: N/A

---

1. (5 points) Define the following:

   - $f(n) = O(g(n))$

     Solution.

     $$f(n) = O(g(n))$$
     $$iff$$
     $$O(g(n)) = f(n) \mid \exists c > 0 \;\&\&\; \exists n_0 > 0$$
     $$0 \leqslant f(n) \leqslant c \cdot g(n)$$
     $$\forall n \geqslant n_0$$

   - $f(n) = o(g(n))$

     Solution.

     $$f(n) = o(g(n))$$
     $$iff$$
     $$o(g(n)) = f(n) \mid \forall c > 0 \;\&\&\; \exists n_0 > 0$$
     $$0 \leqslant f(n) < c \cdot g(n)$$
     $$\forall n \geqslant n_0$$

   - $f(n) = \Omega(g(n))$

     Solution.

     $$f(n) = \Omega(g(n))$$
     $$iff$$
     $$\Omega(g(n)) = f(n) \mid \exists c > 0 \;\&\&\; \exists n_0 > 0$$
     $$0 \leqslant c \cdot g(n) \leqslant f(n)$$
     $$\forall n \geqslant n_0$$

   - $f(n) = \omega(g(n))$

     Solution.

     $$f(n) = \omega(g(n))$$
     $$iff$$
     $$\omega(g(n)) = f(n) \mid \forall c > 0 \;\&\&\; \exists n_0 > 0$$
     $$0 \leqslant c \cdot g(n) < f(n)$$
     $$\forall n \geqslant n_0$$

   - $f(n) = \Theta(g(n))$

     Solution.

     $$f(n) = \Theta(g(n))$$
     $$iff$$
     $$\Theta(g(n)) = f(n) \mid \forall c_1, c_2 > 0 \;\&\&\; \exists n_0 > 0$$
     $$0 \leqslant c_1 \cdot g(n) \leqslant f(n) \leqslant c_2 \cdot g(n)$$
     $$\forall n \geqslant n_0$$

2. (15 points) Find and prove the big-O bound for following recurrences.

(a)

$$B(i) = \begin{cases} 1 & \text{if } i = 1, \\ 2B(i-1)+1 & \text{if } i > 1. \end{cases}$$

Solution.

Iteration

$$B(i) = 2B(i-1)+1$$

$$B(i) = 2(2B(i-2)+1)+1 = 2^2 \cdot B(i-2) + 2 + 1$$

$$B(i) = 2^2 \cdot (2B(i-3)+1) + 2 + 1 = 2^3 \cdot B(i-3) + 2^2 + 1$$

$$B(i) = 2^3 \cdot (2B(i-4)+1) + 2^2 + 1 == 2^4 \cdot B(i-4) + 2^3 + 1$$

Generalized: $B(i) = 2^k \cdot B(i-k) + 2^{k-1} + 1$

$$B(i) = 2^{i-1} \cdot B(1) + 2^{i-2} + 1$$

$$= 2^{i-1} + 2^{i-2} + 1$$

$$= 2^{i-2}(2+1) + 1$$

$$= 3 \cdot 2^{i-2} + 1 \in O(2^i)$$

Geometric Series

$$B(i) = 2B(i-1)+1$$

$$B(1) = 1$$

$$B(2) = 2B(1)+1 = 3$$

$$B(3) = 2B(2)+1 = 7$$

$$B(4) = 2B(3)+1 = 15$$

$$B(5) = 2B(4)+1 = 31$$

Generalized: $B(i) = \sum_{k=1}^{i} 2^{k-1}$

Geometric Series: $B(i) = \sum_{k=1}^{i} 2^{k-1} = S_n = \frac{1-2^n}{-1} = 2^n - 1 \in O(2^n)$

(b)

$$M(i) = \begin{cases} 0 & \text{if } i = 1, \\ 2M(\frac{i}{2}) + i - 1 & \text{if } i \geq 2 \text{ and } i\%2 = 0. \end{cases}$$

Solution.

Master's Theorem:

$$a \cdot T(\frac{n}{b}) = 2 \cdot M(\frac{n}{2})$$

$$a = 2$$

$$b = 2$$

$$f(n) = n - 1 = n$$

$n^{log_2 2}$ *versus* n

n *versus* n

$$M(n) = \Theta(n^{\log_2 2} \cdot \log n) = \Theta(n \cdot \log n) \in O(n \cdot \log n)$$

(c)

$$H(i) = \begin{cases} c & \text{if } i = 1, \\ 2H(i-1) + i & \text{if } i \geq 2. \end{cases}$$

Solution.

Iteration

$$H(i) = 2H(i-1) + i$$

$$H(i) = 2(2H(i-2) + i) + i = 2^2 \cdot H(i-2) + 3i$$

$$H(i) = 2^2 \cdot (2H(i-3) + i) + 3i = 2^3 \cdot H(i-3) + 5i$$

$$H(i) = 2^3 \cdot (2H(i-4) + i) + 5i == 2^4 \cdot H(i-4) + 7i$$

Generalized: $H(i) = 2^k \cdot H(i-k) + (2k-1) \cdot i$

$$B(i) = 2^{k-1} \cdot H(1) + (2(k-1) - 1) \cdot 2$$

$$= 2^{k-1} \cdot c + (2k-3) \cdot 2$$

$$= 2c^{k-1} + 4k - 6 \approx O(2c^i) \approx O(2^i)$$

3. (15 points) Prove the following (you cannot use the Master Theorem).

(a) $T(n) = 6n^2 + 7n$ is $O(n^2)$

Solution.

$$f(n) = O(g(n)) \textit{ iff } f(n) \leqslant c \cdot g(n) \quad \forall n \geqslant k$$

$$\Rightarrow \frac{f(n)}{g(n)} \leqslant c \Rightarrow \frac{6n^2 + 7n}{n^2} \leqslant c$$

Assume k = 1 and c (function bigger than $\frac{f(n)}{g(n)}$*) =* $\frac{6n^2+7n^2}{n^2}$

$$\Rightarrow \frac{6n^2 + 7n}{n^2} \leqslant \frac{6n^2 + 7n^2}{n^2} \quad \forall n \geqslant 1$$

$$\Rightarrow \frac{6n^2 + 7n^2}{n^2} \geqslant \frac{6n^2 + 7n}{n^2} \quad \forall n \geqslant 1$$

$$\Rightarrow 13 \geqslant \frac{6n^2 + 7n}{n^2} \quad \forall n \geqslant 1$$

Thus $f(n) = O(g(n)) \Rightarrow T(n) = 6n^2 + 7n = O(n^2)$

$$\frac{6n^2 + 7n}{n^2} \leqslant 13 \quad \forall n \geqslant 1 \textit{ and for c = 13}$$

$$\Rightarrow 6n^2 + 7n \leqslant 13n^2 \quad \forall n \geqslant 1$$

$$\Rightarrow 7n \leqslant 13n^2 - 6n^2 \quad \forall n \geqslant 1$$

$$\Rightarrow 7n \leqslant 7n^2 \quad \forall n \geqslant 1$$

$$\Rightarrow 7(1) \leqslant 7(1)^2 \quad \forall n \geqslant 1$$

$$\Rightarrow 7 \leqslant 7 \quad \forall n \geqslant 1$$

(b) $T(n) = 4n^2 + 2$ is $\Theta(n^2)$

Solution.

$$f(n) = \Theta(g(n)) \textit{ iff } c_1 \cdot g(n) \leqslant f(n) \leqslant c_2 \cdot g(n) \quad \forall n \geqslant k$$

$$c_1 \cdot g(n) \leqslant f(n) \leqslant c_2 \cdot g(n) \Rightarrow c_1 \cdot n^2 \leqslant 4n^2 + 2 \leqslant c_2 \cdot n^2$$

$$\Rightarrow c_1 \leqslant \frac{4n^2 + 2}{n^2} \leqslant c_2$$

First prove the left hand side (Big Ω):

$$c_1 \leqslant \frac{4n^2 + 2}{n^2}$$

Assume k = 1 and c_1 (function smaller than $\frac{f(n)}{g(n)}$) = $\frac{4(1)+2(1)}{1}$

$$\frac{4(1) + 2(1)}{1} \leqslant \frac{4n^2 + 2}{n^2}$$

$$6 \leqslant \frac{4n^2 + 2}{n^2}$$

Substitute any n value greater than or equal to k (k = 1 as an example)

$$n = 4: \quad 6 \leqslant \frac{4(4)^2 + 2}{(4)^2}$$

$$6 \leqslant \frac{33}{8} \approx 4.125$$

Thus the left hand side has been proven false. Now prove the right hand side (Big O):

$$c_2 \geqslant \frac{4n^2 + 2}{n^2}$$

Assume k = 1 and c_2 (function greater than $\frac{f(n)}{g(n)}$) = $\frac{4(n^2)+2(n^2)}{n^2}$

$$\frac{4(n^2) + 2(n^2)}{n^2} \geqslant \frac{4n^2 + 2}{n^2}$$

$$\frac{4(n^2) + 2(n^2)}{n^2} \geqslant \frac{4n^2 + 2}{n^2}$$

Substitute any n value greater than or equal to k (k = 1 as an example)

$$n = 4: \quad \frac{4((4)^2) + 2((4)^2)}{(4)^2} \geqslant \frac{4(4)^2 + 2}{(4)^2}$$

$$6 \geqslant \frac{33}{8} \approx 4.125$$

Thus the right hand side has been proven false. The definition does not hold which means that $T(n) = 4n^2 + 2$ is NOT $\Theta(n^2)$

(c) $T(n) = 1^2 + 2^2 + 3^2 + \ldots + n^2$ is $\Theta(n^3)$

Solution.

$$T(n) = 1^2 + 2^2 + 3^2 + \ldots + n^2 = \sum_{i=1}^{n} i^2$$

$$\sum_{i=1}^{n} i^2 = \sum_{i=1}^{n} n^2 \textit{ since } i \leqslant n$$

$$\sum_{i=1}^{n} n^2 = n^2 \sum_{i=1}^{n} 1 = n^3 \textit{ since } \sum_{i=1}^{n} 1 = n$$

$$\approx \Theta(n^3)$$

Thus $T(n) = 1^2 + 2^2 + 3^2 + \ldots + n^2$ *is* $\Theta(n^3)$ *has been proven true.*

4. (6 pts) Prove or disprove 3 asymptotic implications: If $f(n)$ is in $O(g(n))$ then is it always true that:

   (a) $\log_2(f(n))$ is in $O((\log_2(g(n))$.

   Solution. *Definition of O(g(n)):*

   $$f(n) \leqslant c \cdot g(n)$$

   Assume $f(n) = 2 \cdot n^2$ *and* $g(n) = n^2$*:*

   $$\log_2 2 \cdot n^2 \leqslant \log_2 n^2$$

   $$\log_2 2 \cdot \log_2 n^2 \leqslant \log_2 n^2$$

   $$1 + \log_2 n^2 \leqslant c + \log_2 n^2$$

   Thus $\log_2(f(n))$ *is in* $O((\log_2(g(n))$ *has been proven true.*

(b) $2^{f(n)}$ is in $O(2^{g(n)})$.

Solution. *Definition of O(g(n)):*

$$f(n) \leqslant c \cdot g(n)$$

Assume $f(n) = 2 \cdot \log n$ *and* $g(n) = \log n$ *and log base 2:*

$$2^{2 \cdot \log n} \leqslant 2^{\log n}$$

$$n^2 \leqslant n$$

Thus $2^{f(n)}$ *is in* $O(2^{g(n)})$ *has been proven false.*

(c) $f(n)^2$ is in $O(g(n)^2)$.

Solution. *Definition of O(g(n)):*

$$f(n) \leqslant c \cdot g(n)$$

Assume $f(n) = n$ and $g(n) = n$:

$$n^2 \leqslant c \cdot n^2$$

As $f(n)$ and $g(n)$ are increasing as long as constant c is a significantly greater number the square of a function and its respective $g(n)$ will scale accordingly. $f(n)^2$ is in $O(g(n)^2)$ is valid.

5. (4 points) Write the pseudocode for insertion sort. What is the worst case number of (a) assignments and (b) comparisons made?

```
//Insertion Sort
insertionSort(arr)
   for every element with index i in arr
   val = arr[i]
   j = i-1
   while j >= 0 and arr[j] > key
      arr[j+1] = arr[j]
      j--
   arr[j+1]=key
end
```

Solution. *Worst case for this algorithm is the array is in the complete reverse order of the targeted order. The worst case runtime is $O(n^2)$ and the best case is $O(n)$. In terms of assignments, the algorithm (worst case) will complete n-1 array assignments in the first run then n-2 and onwards it is close to $\frac{n^2}{2}$ array assignments. In terms of comparisons, the result is $\frac{n^2}{2}$ as once the algorithm finds a proper place to put the value it will stop comparing.*

6. (5 points) You are presented with two sorting algorithms. You have to select one of them. The documentation provided states that both of these algorithms perform $O(n^2)$ comparisons in the worst case. You want to build a fast program and therefore want to optimise speed. Does it matter which algorithm you select? How would you make the decision?

   Solution. *Of the two, I would choose the one with the fastest runtime (runtime != complexity) and the one with the least memory used during runtime as that makes the algorithm more effiecient releasing resources for other possible asyncronous functions running at the same time. If there is no worry on memory based resources then either algorithm should be fine as long as the average case bigO and best case bigO is the same.*